IS 1210
Database Systems

Lecturer 1 – Normalization (Part 1)

MVPT Lakshika,

Lecturer (Probationary),

Department of Information Systems Engineering, UCSC.

Contact: tlv@ucsc.cmb.ac.lk

Course Description

Number of Credits: 3 Credits (2L,1P)

Evaluation Criteria: Continuous Assessments: 30%, Final Examination: 70%

Assessment Criteria:

- One (2) In-class Assessment
- One (1) Practical Assessment

(Dates will be announced)

- Several Unannounced In-class quizzes
- Several Unannounced practical quizzes

Practical Sessions:

Software Tools

- Windows Command-Line Interface (CLI)
- Microsoft Access
- MS SQL Server

Learning Outcomes

Upon completion of this course, students will be able to do the following:

LO1. Describe the role of a database system and the functions of a database administrator.

LO2. Explain the ANSI-SPARC three-schema architecture for databases and differentiate between conceptual, external, and internal schemas.

LO3. Define fundamental concepts of the relational model, including relations, attributes, domains, keys, foreign keys, entity integrity, and referential integrity.

LO4. Analyze database requirements and evaluate the suitability of different database designs.

LO5. Create Entity-Relationship diagrams for given scenarios and translate conceptual models into relational schemas.

LO6. Demonstrate the process of normalization and apply normalization techniques to relations.

LO7. Formulate SQL queries of varying complexity to retrieve and manipulate data.

LO8. Define the concept of views in database systems, create data views using SQL, and evaluate the advantages and disadvantages of views.

LO9. Develop database applications using SQL and other relevant tools and technologies.

Recommended Textbooks

- Elmasri, R. and Navathe, S.B., 2015. *Fundamentals of database systems*. 7th Edition, Addison-Wesley.
- Ramakrishnan, R. and Gehrke, J., 2003. *Database management systems* (Vol. 3). New York: McGraw-Hill.
- DeBarros, A., 2022. Practical SQL: A Beginner's Guide to Storytelling with Data. No Starch Press.

Course Content

1. Introduction to data normalization and normal forms (Background)
    - What is normalization
    - Benefits of normalization
    - Normalization Rules
2. First Normal Form (1NF)
3. Second Normal Form (2NF)
4. Third Normal Form (3NF)
5. Boyce-Codd Normal Form (BCNF)
6. Normalization Considerations

Outline of this lecture

1. What is normalization?
2. Data Redundancy
3. Update Anomalies

1. What is Normalization?

Background

What is the main objective of designing a database for an enterprise?

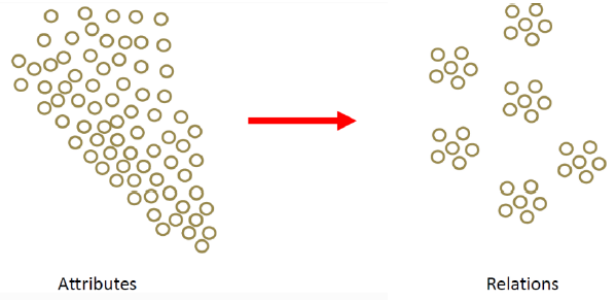
- Create an accurate representation of the data, relationships between the data, and constraints on the data that are relevant to the enterprise.

To help achieve this objective, you can use one or more database design techniques.

Ex: ER modelling, Normalization

Normalization

- Normalization is a technique for producing a set of relations with desirable properties, given the data requirements of an enterprise.
- Normalization begins by examining the relationships (called functional dependencies) between attributes.
  - Attributes describe some property of the data or of the relationships between the data.



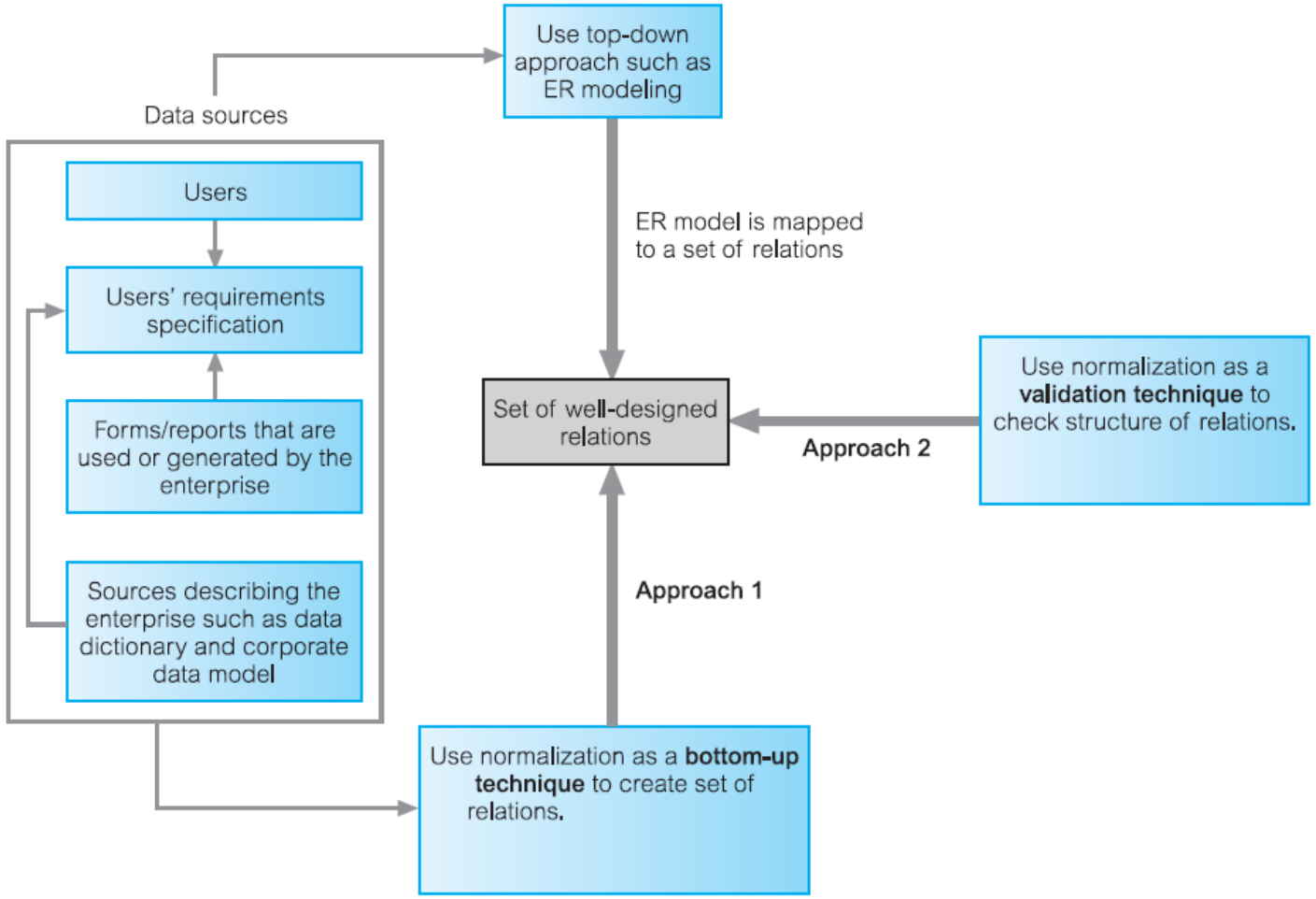
- Normalization uses a series of tests (described as normal forms) to help identify the optimal grouping for these attributes to ultimately identify a set of suitable relations that support the data requirements.

Purpose of Normalization

- Identify a suitable set of relations that support the data requirements of an enterprise.
- The characteristics of a suitable set of relations include the following:
  - the minimal number of attributes necessary to support the data requirements.
  - attributes with a close logical relationship (functional dependency) are found in the same relation.
  - minimal redundancy with each attribute represented only once, with the important exception of attributes that form all or part of foreign keys, which are essential for the joining of related relations.

Benefit: The database will be easier for the user to access and maintain the data, and take up minimal storage.

How Normalization Supports Database Design?



- Which approach to follow?

 - Depends on the size and complexity of the database being described by the data sources.

- Rely on the preference and expertise of the database designer.

- **Approach 1** is often limited by the level of detail that the database designer is reasonably expected to manage.
- In **Approach 2,** the database designer focuses on only part of the database, such as a single relation, at any one time.
- Therefore, no matter what the size or complexity of the database, normalization can be usefully applied.

2. Data Redundancy

What is data redundancy ?

- A major aim of relational database design is to group attributes into relations to minimize data redundancy.
  - Redundancy occurs when the same piece of data is stored in multiple places within a database.
- If this aim is achieved, the potential benefits for the implemented database include the following;
  - **updates to the data stored in the database are achieved with a minimal number of operations**, thus reducing the opportunities for data inconsistencies occurring in the database;
  - **reduction in the file storage space** required by the base relations thus minimizing costs.
- Of course, relational databases also rely on the existence of a certain amount of data redundancy. This redundancy is in the form of copies of primary keys (or candidate keys) acting as foreign keys in related relations to enable the modeling of relationships between data.

Problems associated with data redundancy

Activity: Compare the Staff and Branch relations with the StaffBranch relation.

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Staff (staffNo, sName, position, salary, branchNo)
Branch (branchNo, bAddress)
StaffBranch (staffNo, sName, position, salary, branchNo, bAddress)

Problems associated with data redundancy

Activity: Compare the Staff and Branch relations with the StaffBranch relation.

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

StaffBranch relation: there is redundant data; the details of a branch are repeated for every member of staff located at that branch.

Branch relation: branch details appear only once for each branch in the Branch relation.

Staff relation: only the branch number (branchNo) is repeated in the Staff relation to represent where each member of staff is located.

Relations that have redundant data may have problems called **update anomalies**.

3. Update Anomalies

Insertion, Deletion, and Modification anomalies

Insertion anomaly - example

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Task: Insert the details of new members of staff located at branch number B007 into the StaffBranch relation.

- Here, you must enter the correct details of branch number B007 so that the branch details are consistent with values for branch B007 in other tuples of the StaffBranch relation.

The Staff and Branch relations do not suffer from this potential inconsistency because you enter only the appropriate branch number for each staff member in the Staff relation.

Instead, the details of branch number B007 are recorded in the database as a single tuple in the Branch relation.

Insertion anomaly - example

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Task: Insert details of a new branch that currently has no members of staff into the StaffBranch relation.

- It is necessary to enter *nulls* into the attributes for staff, such as staffNo. However, as staffNo is the primary key for the StaffBranch relation, attempting to enter nulls for staffNo violates entity integrity and is not allowed.

The Staff and Branch relations avoids this problem because branch details are entered in the Branch relation separately from the staff details.

The details of staff located at that branch are entered at a later date into the Staff relation.

Insertion anomaly - Summary

- An insertion anomaly occurs when data cannot be inserted into a database due to other missing data.
- This is most common for fields where a foreign key must not be NULL, but lacks the appropriate data.

Example: Consider a simple user database.

  - A user must have a *groupID* as a foreign key.
  - No groups have yet been created.
  - Thus, a user can not be inserted into the database as the *groupID* must not be NULL.

- This insertion anomaly itself is a form of data integrity problem.
- This situation highlights the importance of order of operations when inserting related data, especially when foreign key constraints are involved.

Deletion anomaly - example

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Task: Delete a tuple from the StaffBranch relation that represents the member of staff with staff number SA9 (Mary Howe).

- When you delete the tuple for staff number SA9 (Mary Howe) from the StaffBranch relation, the details relating to branch number B007 are lost from the database.

The Staff and Branch relations avoids this problem, because branch tuples are stored separately from staff tuples and only the attribute branchNo relates the two relations.

- If you delete the tuple for staff number SA9 from the Staff relation, the details on branch number B007 remain unaffected in the Branch relation.

Deletion anomaly - Summary

- A deletion anomaly occurs when data is unintentionally lost due to the deletion of other data.

Example: if a database row contained "**Username**" and "**User Group**“

  - John and Fred are in the user group "*Contributors*“.
  - If John and Fred are removed from the database, the *Contributors* group will also disappear.
  - This is because we haven't normalised our data, meaning the only reference to the *Contributors* user group lies within the same database row.

- Hence, removing the only two references of our user group results in the loss of data accuracy and integrity.
- This also goes to show why it's important for us to normalise our data and how combining unlike information can be problematic.

Modification anomaly - example

StaffBranch

staffNo	sName	position	salary	branchNo	bAddress
SL21	John White	Manager	30000	B005	22 Deer Rd, London
SG37	Ann Beech	Assistant	12000	B003	163 Main St, Glasgow
SG14	David Ford	Supervisor	18000	B003	163 Main St, Glasgow
SA9	Mary Howe	Assistant	9000	B007	16 Argyll St, Aberdeen
SG5	Susan Brand	Manager	24000	B003	163 Main St, Glasgow
SL41	Julie Lee	Assistant	9000	B005	22 Deer Rd, London

Staff

staffNo	sName	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007
SG5	Susan Brand	Manager	24000	B003
SL41	Julie Lee	Assistant	9000	B005

Branch

branchNo	bAddress
B005	22 Deer Rd, London
B007	16 Argyll St, Aberdeen
B003	163 Main St, Glasgow

Task: Change the value of the address attribute of branch number B003 in the StaffBranch relation.

- If this modification is not carried out on all the appropriate tuples of the StaffBranch relation, the database will become inconsistent.

The Staff and Branch relations have more desirable properties than the StaffBranch relation.

We can avoid these anomalies by decomposing the original relation into the appropriate sub-relations.

Modification anomaly - Summary

- An update anomaly occurs when data is only partially updated in a database.
- A database that hasn't undergone normalization may reference the same data element in more than one location.
- As these locations haven't been consolidated and referenced, we have to make sure each location is manually updated.
- This can cause problems as we then need to spend time searching for and updating each reference to the data element.

Decomposition of a larger relation into smaller relations

Decomposition should have below two properties:

- The lossless-join property ensures that any instance of the original relation can be identified from corresponding instances in the smaller relations.
- The dependency preservation property ensures that a constraint on the original relation can be maintained by simply enforcing some constraint on each of the smaller relations. In other words, we do not need to perform joins on the smaller relations to check whether a constraint on the original relation is violated (Or Simply preserve the associated set of functional dependencies).

  -Although the **StaffBranch** relation is subject to update anomalies, we can avoid these anomalies by decomposing the original relation into the **Staff** and **Branch** relations.

Decomposition of a larger relation into smaller relations

Let us populate R with sample data and try the experiment;

A	B	C	D
a_1	a_2	a_3	a_4
a_1	a_4	a_3	a_2

According to the decomposition, we shall get R_1 and R_2 as follows;

R_1		
A	B	C
a_1	a_2	a_3
a_1	a_4	a_3

R_2		
A	C	D
a_1	a_3	a_4
a_1	a_3	a_2

Join back R_1 and R_2 must result in R if the decomposition is lossless.

$R_1 \bowtie R_2 = R'$

A	B	C
a_1	a_2	a_3
a_1	a_4	a_3

$\bowtie$

A	C	D
a_1	a_3	a_4
a_1	a_3	a_2

$=$

A	B	C	D
a_1	a_2	a_3	a_4
a_1	a_2	a_3	a_2
a_1	a_4	a_3	a_2
a_1	a_4	a_3	a_4

R′ is the result of natural join of R_1 and R_2, and R′ is not equal to R the base relation. Hence, **the decomposition is not lossless join decomposition**.

Decomposition of a larger relation into smaller relations

R

A	B	C
1	x	p
2	y	q

decomposition →

After decomposition

R1(A, B)

A	B
1	x
2	y

R2(A, C)

A	C
1	p
2	q

Next, Join $R_1 \bowtie R_2$

R

A	B	C
1	x	p
2	y	q

This is exactly the original table.
Lossless Join is satisfied.

Wrap Up

What is Normalization

Purpose of Normalization

Problems associated with data redundancy

Update anomalies (Insertion, Deletion, and Modification anomalies)

Any Questions?